Progress Report #1 — RAG Tutoring System

Sachet Kanchugar | Akash Deep Singh | Sanjeet Srinivasa
Foundations of AI | March 12, 2026

What Has Already Been Achieved

Significant progress has been made on both the backend and frontend independently. The backend is built with **FastAPI** and structured across dedicated modules: `main.py` (routes), `rag.py` (retrieval and generation), `document_loader.py` (parsing and chunking), `vector_store.py` (FAISS index), and `auth.py` (token verification). It exposes three endpoints — `POST /upload`, `GET /documents`, and `POST /chat` — with teacher-only upload protection designed around an `Authorization: Bearer` header checked against `TEACHER_SECRET` in `.env`. PDFs are parsed with `PyPDFLoader` and text files with `TextLoader`; content is chunked via `RecursiveCharacterTextSplitter` (`chunk_size=500`, `overlap=100`), embedded with `text-embedding-3-small`, and stored in a local **FAISS** index. The RAG pipeline retrieves the top-3 chunks per query and passes them to `gpt-4o-mini`, returning structured JSON with an answer and source metadata (document name, page, snippet). The frontend, built with **React** and **TailwindCSS**, includes `UploadPage.js` for teacher document management and `ChatPage.js` for the student Q&A interface. A `requirements.txt` and `.env.example` are provided for local setup.

Immediate Next Steps

1. Complete frontend–backend integration by wiring all API calls (`/upload`, `/documents`, `/chat`) from the React UI to the FastAPI server with proper error handling and state management.
2. Implement the full authentication flow on the frontend, including a teacher token input, role-based view separation, and secure passing of the `Authorization: Bearer` header on upload requests.
3. Integrate structure-aware PDF parsing (e.g., `pymupdf` or `unstructured`) to better handle tables, equations, and slide layouts that plain text extraction misses.

Challenges and Adjustments

Integration and Authentication. The frontend and backend have been developed in parallel but are not yet connected. Wiring the React UI to the FastAPI server requires consistent error handling and state synchronization across all three flows (upload, document listing, chat). In parallel, while the backend supports bearer token authentication, the frontend has no login mechanism yet — the teacher token input, protected routing, and role-based view separation all remain to be implemented.

Multimodal Content. Pure text-based RAG degrades for course materials that rely on diagrams, equations, code screenshots, and complex slide layouts. Standard PDF extraction either drops visual content or flattens it into low-quality text (broken LaTeX, isolated captions), causing retrieval to miss core teaching material. The project scope is adjusted to include a multimodal augmentation layer using structure-aware parsing and GPT-4o vision-based image descriptions.

Evaluation Complexity. There is no simple pass/fail test for RAG quality — failures can stem from poor retrieval, LLM misinterpretation, or genuinely missing content, and these are difficult to distinguish without structured measurement. A lightweight labeled Q&A test set derived from uploaded course materials will be constructed to enable systematic evaluation of retrieval relevance and answer correctness.

Overall Plan for the Next Month

Week	Focus	Tasks
Week 1	**Integration & Auth**	Connect all API endpoints (`/upload`, `/documents`, `/chat`) to the React UI; implement teacher token input, protected routing, and role-based view separation.
Week 2	**Retrieval Quality**	Integrate structure-aware PDF parsing (`pymupdf` / `unstructured`); add GPT-4o vision for image descriptions; re-index test documents with updated pipeline.
Week 3	**Evaluation & UI Polish**	Build a labeled Q&A test set; tune chunk size and retrieval parameters; polish frontend (loading states, error handling, citation display).
Week 4	**Testing & Submission**	End-to-end testing, bug fixes, final documentation, and submission preparation.

The core RAG pipeline and UI components are in place; the remaining work centers on integration, authentication, and retrieval quality improvements.